

P2P – III

Lecture 6

January 21, 2026

Find the bug, if any

<https://cse.iitk.ac.in/users/cs633/2025-26-2/bug1.c>

```
// initialize data
for (int i=0; i<count; i++)
    buf[i] = myrank+i;

if (myrank == 0) {
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD);
    printf ("%d %d %d\n", myrank, buf[0], buf[1]);
}
else
    if (myrank == 1) {
        MPI_Recv (&buf[count-1], count, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);
        printf ("%d %d %d\n", myrank, buf[count-1], buf[count]);
    }
```

Execution of bug1.c

```
class$ mpirun -np 2 ./bug1 10 20
0 0 1
1 0 1
class$ mpirun -np 2 ./bug1 100 20
0 0 1
```

```
=====
= BAD TERMINATION OF ONE OF YOUR APPLICATION PROCESSES
= PID 433780 RUNNING AT csews21
= EXIT CODE: 139
= CLEANING UP REMAINING PROCESSES
= YOU CAN IGNORE THE BELOW CLEANUP MESSAGES
=====
YOUR APPLICATION TERMINATED WITH THE EXIT STRING: Segmentation fault (signal 11)
This typically refers to a problem with your application.
Please see the FAQ page for debugging suggestions
```

Code Snippet - bug2.c

```
// initialize data
for (int i=0; i<count_; i++)           // Or, initialize to count
    buf[i] = myrank+i;

if (myrank == 0) {
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD);
    printf ("%d %d %d\n", myrank, buf[0], buf[1]);
}
else
    if (myrank == 1) {
        MPI_Recv (&buf[count_], count, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);
        printf ("%d %d %d\n", myrank, buf[count_], buf[count_+1]);
    }
```

Executing bug2.c

```
class$ mpirun -np 2 ./bug2 28 2
```

```
0 0 1
```

```
1 0 1
```

```
class$ mpirun -np 2 ./bug2 28 4
```

```
0 0 1
```

```
1 0 1
```

```
class$ mpirun -np 2 ./bug2 28 6
```

```
0 0 1
```

```
1 0 1
```

Check the received count

```
// initialize data
for (int i=0; i<count; i++)
    buf[i] = myrank+i;

if (myrank == 0) {
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD);
    printf ("%d %d %d\n", myrank, buf[0], buf[1]);
}
else
    if (myrank == 1) {
        MPI_Recv (&buf[count_], count, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);
        int recvcount; MPI_Get_count (&status, MPI_INT, &recvcount);
        printf ("%d %d %d %d\n", myrank, recvcount, buf[count_], buf[count_+1]);
    }
```

Output

```
mpirun -np 2 ./bug2 25 4
0 0 1
1 25 0 1
class$ mpirun -np 2 ./bug2 30 4
0 0 1
1 30 0 1
class$ mpirun -np 2 ./bug2 30 10
0 0 1
28 30 0 1
class$ mpirun -np 2 ./bug2 30 5
0 0 1
1 30 0 1
```

Executing bug2.c

```
class$ mpirun -np 2 ./bug2 28 8
0 0 1
25708 0 1
class$
class$ mpirun -np 2 ./bug2 100 20
0 0 1
```

```
=====
= BAD TERMINATION OF ONE OF YOUR APPLICATION PROCESSES
= PID 3884421 RUNNING AT cseproj132
= EXIT CODE: 139
= CLEANING UP REMAINING PROCESSES
= YOU CAN IGNORE THE BELOW CLEANUP MESSAGES
=====
```

```
YOUR APPLICATION TERMINATED WITH THE EXIT STRING: Segmentation fault (signal 11)
This typically refers to a problem with your application.
Please see the FAQ page for debugging suggestions
```


Bug fix?

```
// initialize data
for (int i=0; i<count_; i++)
    buf[i] = myrank+i;

if (myrank == 0) {
    MPI_Send (buf, count_, MPI_INT, 1, 1, MPI_COMM_WORLD);
    printf ("%d %d %d\n", myrank, buf[0], buf[1]);
}
else
    if (myrank == 1) {
        MPI_Recv (&buf[count_], count, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);
        int recvcnt; MPI_Get_count (&status, MPI_INT, &recvcnt);
        printf ("%d %d %d\n", myrank, buf[count_], buf[count_+1]);
    }
```

Output

Programmer's
responsibility to correctly
specify receiver buffer
location, use asserts, etc.

```
class$ mpirun -np 2 ./bug2_fix 10 2
0 0 1
1 0 1
class$ mpirun -np 2 ./bug2_fix 10 5
0 0 1
1 0 1
class$ mpirun -np 2 ./bug2_fix 10 7
0 0 1
1 0 1
class$ mpirun -np 2 ./bug2_fix 10 8
0 0 1
1 0 1
class$ mpirun -np 2 ./bug2_fix 20 8
0 0 1
1 0 1
class$ mpirun -np 2 ./bug2_fix 20 15
0 0 1
11 0 1
```

Doubt (from Jan 14 lecture slide 8 or 9?)

The mesh has 4 hosts and $\text{ppn}=2$, if there are two communications happening over the same link, specifically for the communication step1 for the round robin case, #hops are written 1,1 but the communication pattern 1->0 and 3->2 are over the same link between host 2 and host 1. Why are we counting it as a single hop? like the communicating processes are two sending their data over a single link, so if we count them as a single hop, are we considering them to be sending data in parallel? because at a given point in time, explicit communication would imply sending data from one process at a time, otherwise, two processes sending data over (i) two different channels and (ii) a single channel at a time (in terms of #hops) kind of becomes the same? because if that assumption follows, the 2d mesh homework question gives same answers for sequential and round robin (slide 9) as well as that for linear network topology with cases of 8 processes $\text{ppn}=2$ and 16 processes $\text{ppn}=4$ all of them happen to give the same answer, it would have been different if we would consider the single link communication to be process wise dependent, because for 16 processes, essentially the communication steps have the neighbouring communications increased which lead to increase in the type of hops that we are counting to be the same over a single link.

Number of Hops on 4 Processes (Jan 14, slide 8)



Communication step 1: 1 → 0, 3 → 2
Communication step 2: 2 → 0



Sequential placement

Communication step 1 #hops: 0, 0

Max: 0

Communication step 2 #hops: 1

Max: 1

Sum: 1



Round-robin placement

Communication step 1 #hops: 1, 1

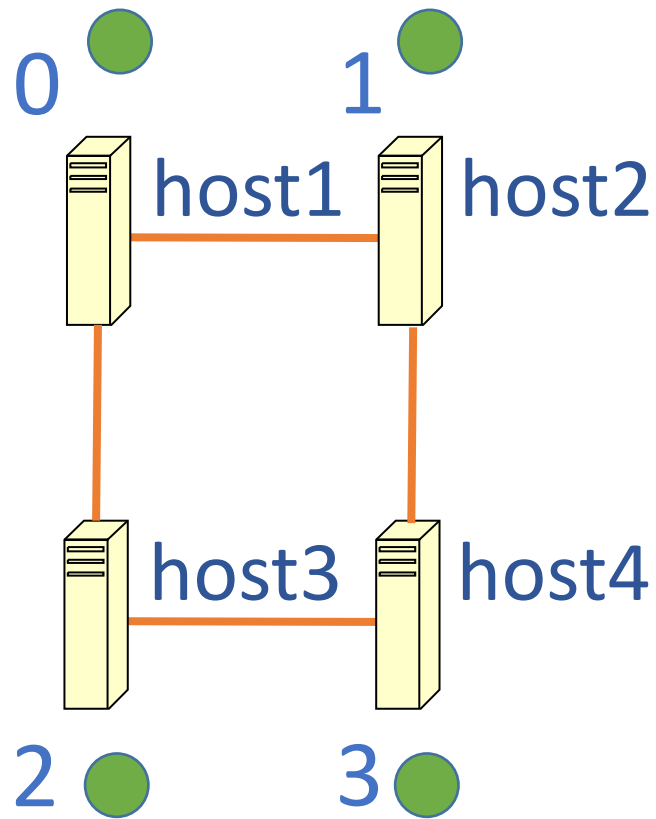
Max: 1

Communication step 2 #hops: 0

Max: 0

Sum: 1

Parallel Sum on 4 processes in a 2D Mesh (Jan 14, slide 9)



Communication step 1: 1 $\rightarrow$ 0, 3 $\rightarrow$ 2
Communication step 2: 2 $\rightarrow$ 0

Sequential placement

Communication step 1 #hops: 1, 1

Communication step 2 #hops: 1

Max: 1

Max: 1

Sum: 2

Send – Recv Example (sendrecv1.c)

```
if (myrank == 0) {  
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD);  
    MPI_Recv (buf, count, MPI_INT, 1, myrank, MPI_COMM_WORLD, &status);  
}  
else if (myrank == 1) {  
    MPI_Recv (buf, count, MPI_INT, 0, myrank, MPI_COMM_WORLD, &status);  
    MPI_Send (buf, count, MPI_INT, 0, 0, MPI_COMM_WORLD);  
}  
printf ("%d %d %d\n", myrank, count, status.MPI_SOURCE);
```

Send – Recv Example (sendrecv2.c)

```
if (myrank == 0) {  
    MPI_Recv (buf, count, MPI_INT, 1, myrank, MPI_COMM_WORLD, &status);  
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD);  
}  
else if (myrank == 1) {  
    MPI_Send (buf, count, MPI_INT, 0, 0, MPI_COMM_WORLD);  
    MPI_Recv (buf, count, MPI_INT, 0, myrank, MPI_COMM_WORLD, &status);  
}  
printf ("%d %d %d\n", myrank, count, status.MPI_SOURCE);
```

Send – Recv Example

Incorrect code – relies on MPI implementation and value of count

```
if (myrank == 0) {  
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD);  
    MPI_Recv (buf, count, MPI_INT, 1, myrank, MPI_COMM_WORLD, &status);  
}  
else if (myrank == 1) {  
    MPI_Send (buf, count, MPI_INT, 0, 0, MPI_COMM_WORLD);  
    MPI_Recv (buf, count, MPI_INT, 0, myrank, MPI_COMM_WORLD, &status);  
}  
printf ("%d %d %d\n", myrank, count, status.MPI_SOURCE);
```


Order of Invocation Matters

0	1	
MPI_Send MPI_Send	MPI_Recv MPI_Recv	Safe
MPI_Send MPI_Recv	MPI_Send MPI_Recv	Unsafe
MPI_Send MPI_Recv	MPI_Recv MPI_Send	Safe
MPI_Recv MPI_Send	MPI_Recv MPI_Send	Unsafe

Note:

0 sends to 1 and 1 sends to 0

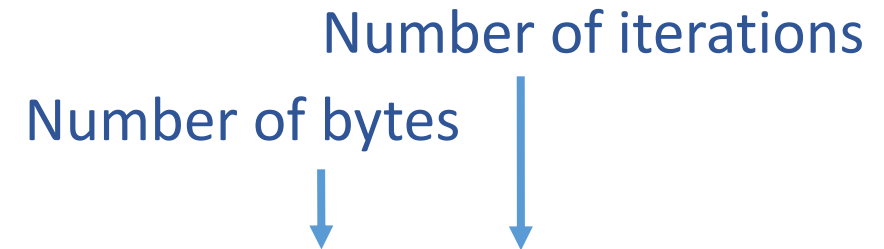
0 receives from 1 and 1 receives from 0

Eager vs. Rendezvous Protocol

- Eager
 - Send completes without acknowledgement from destination
 - `MPIR_CVAR_CH3_EAGER_MAX_MSG_SIZE` (check output of `mpivars`)
 - Small messages, typically 128 KB (at least in MPICH)
 - Not safe to use the send buffer if message size is small
- Rendezvous
 - Requires an acknowledgement from a matching receive
 - Large messages

Eager vs. Rendezvous Example

Number of iterations
Number of bytes



```
class $ mpirun -np 2 -hosts csews1:1,csews30:1 ./send 10000 1000
Rank 0 (csews1) 0.085077
Rank 1 (csews30) 0.095820
class $ mpirun -np 2 -hosts csews1:1,csews30:1 ./send 10000 1000
Rank 0 (csews1) 0.079437
Rank 1 (csews30) 0.092750
class $ export MPIR_CVAR_CH3_EAGER_MAX_MSG_SIZE=1000
class $ mpirun -np 2 -hosts csews1:1,csews30:1 ./send 10000 1000
Rank 0 (csews1) 0.499451
Rank 1 (csews30) 0.499622
class $ mpirun -np 2 -hosts csews1:1,csews30:1 ./send 10000 1000
Rank 1 (csews30) 0.503129
Rank 0 (csews1) 0.503376
```

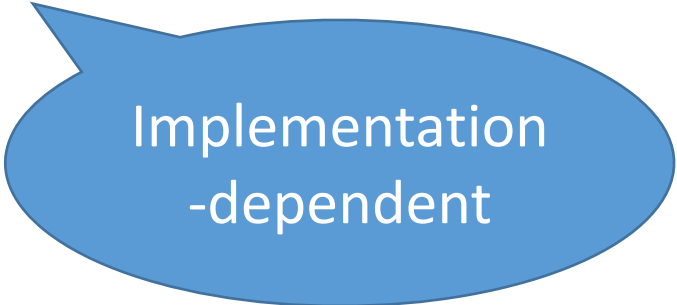
Simple code: Rank 0 sends 10000 integers to rank 1 1000 times

Standard Mode Send (MPI_Send)

“blocking: it does not return until the message data and envelope have been safely stored away so that the sender is free to modify the send buffer”

Standard Mode Send

- A standard mode send **can** be started whether or not a matching receive has been posted.
- It **may** complete before a matching receive is posted.
- It is up to MPI to decide whether outgoing messages will be buffered. MPI **may** buffer outgoing messages.
- MPI takes the position that correct (and therefore, portable) programs do not rely on system buffering in standard mode.



Implementation
-dependent

Other Send Communication Modes

- MPI_Send Standard
 - May complete before matching receive is posted (implementation dependent)
- MPI_Bsend Buffered
 - May complete before matching receive is posted
 - Does not depend on the occurrence of a matching receive
- MPI_Ssend Synchronous
 - Completes only if a matching receive is posted
- MPI_Rsend Ready
 - Started only if a matching receive is posted

MPI_BUFFER_ATTACH/DETACH

MPI_BUFFER_ATTACH (buffer_addr, size)

- Provides to MPI a buffer in the user's memory
- Used by messages sent in buffered mode
- Only one buffer can be attached to a process at a time

MPI_BUFFER_DETACH (buffer_addr, size)

Bsend Example

bsend.c

```
// Only one buffer can be attached to a process at a time
MPI_Buffer_attach(buf, bufsize);

start_time = MPI_Wtime ();
if (myrank == 0){
    MPI_Bsend(arr, BUFSIZE/2, MPI_INT, myrank+1, myrank+1, MPI_COMM_WORLD);
    MPI_Bsend(arr+BUFSIZE/2, BUFSIZE/2, MPI_INT, myrank+2, myrank+2, MPI_COMM_WORLD);
}
else
    MPI_Recv(recvarr, BUFSIZE/2, MPI_INT, 0, MPI_ANY_TAG, MPI_COMM_WORLD, &status);

if (myrank == 0) printf ("Rank %d: time= %lf\n", myrank, MPI_Wtime() - start_time);
else printf ("%d\n", recvarr[0]);

MPI_Buffer_detach(&buf, &bufsize);
```

mpirun -np 3 ./bsend 20000

MPI_Bsend

The size given should be the sum of the sizes of all outstanding Bsend's that you intend to have, plus 'MPI_BSEND_OVERHEAD' for each Bsend that you do.

```
// Only one buffer can be attached to a process at a time
MPI_Buffer_attach(buf, bufsize);

start_time = MPI_Wtime ();
if (myrank == 0){
    MPI_Bsend(arr, BUFSIZE/2, MPI_INT, myrank+1, myrank+1, MPI_COMM_WORLD);
    MPI_Bsend(arr+BUFSIZE/2, BUFSIZE/2, MPI_INT, myrank+2, myrank+2, MPI_COMM_WORLD);
}
else
    MPI_Recv(recvarr, BUFSIZE/2, MPI_INT, 0, MPI_ANY_TAG, MPI_COMM_WORLD, &status);

if (myrank == 0) printf ("Rank %d: time= %lf\n", myrank, MPI_Wtime() - start_time);
else printf ("%d\n", recvarr[0]);

MPI_Buffer_detach(&buf, &bufsize);
```

Example

Example 3.6. An example of nonovertaking messages.

```
CALL MPI_COMM_RANK(comm, rank, ierr)
IF (rank .EQ. 0) THEN
    CALL MPI_BSEND(buf1, count, MPI_REAL, 1, tag, comm, ierr)
    CALL MPI_BSEND(buf2, count, MPI_REAL, 1, tag, comm, ierr)
ELSE IF (rank .EQ. 1) THEN
    CALL MPI_RECV(buf1, count, MPI_REAL, 0, MPI_ANY_TAG, comm, status, &
                  ierr)
    CALL MPI_RECV(buf2, count, MPI_REAL, 0, tag, comm, status, ierr)
END IF
```

The message sent by the first send must be received by the first receive, and the message sent by the second send must be received by the second receive.

Example

Example 3.7. An example of two, intertwined matching pairs.

```
CALL MPI_COMM_RANK(comm, rank, ierr)
IF (rank .EQ. 0) THEN
    CALL MPI_BSEND(buf1, count, MPI_REAL, 1, tag1, comm, ierr)
    CALL MPI_SSEND(buf2, count, MPI_REAL, 1, tag2, comm, ierr)
ELSE IF (rank .EQ. 1) THEN
    CALL MPI_RECV(buf1, count, MPI_REAL, 0, tag2, comm, status, ierr)
    CALL MPI_RECV(buf2, count, MPI_REAL, 0, tag1, comm, status, ierr)
END IF
```

Many-to-one Revision

All ranks send to the last rank

```
MPI_Comm_rank (MPI_COMM_WORLD, &myrank);  
MPI_Comm_size (MPI_COMM_WORLD, &size);
```

Code (Many-to-one)

sendmodes.c

```
start_time = MPI_Wtime ();
if (myrank < size-1)
{
    if (option == 1)
        MPI_Send(arr, BUFFER, MPI_INT, size-1, myrank, MPI_COMM_WORLD);
    else if (option == 2)
        MPI_Bsend(arr, BUFFER, MPI_INT, size-1, myrank, MPI_COMM_WORLD);
    else if (option == 3)
        MPI_Ssend(arr, BUFFER, MPI_INT, size-1, myrank, MPI_COMM_WORLD);
}
else
{
    int count, recvarr[size][BUFFER];
    for (int i=0; i<size-1; i++)
        MPI_Recv(recvarr[i], BUFFER, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
}
```

Timing (Send Modes)

Data size:
200000000 integers

```
Rank 0: time= 0.808475
Rank 1: time= 0.806481

Rank 0: time= 0.803519
Rank 1: time= 0.792499

Rank 0: time= 0.386989
Rank 1: time= 0.812923

Rank 0: time= 0.390736
Rank 1: time= 0.813753

Rank 0: time= 0.805903
Rank 1: time= 0.802252

Rank 0: time= 0.795828
Rank 1: time= 0.797992
```

MPI_Send

MPI_Bsend

MPI_Ssend

Performance of Send Modes (Many-to-one)

```
class mpirun -np 4 -hosts csews1,csews2 ./sendmodes 1000 1
Rank 3: time= 0.000509
class mpirun -np 4 -hosts csews1,csews2 ./sendmodes 1000 2
Rank 3: time= 0.000530
class mpirun -np 4 -hosts csews1,csews2 ./sendmodes 1000 3
Rank 3: time= 0.006357
```

MPI_Send

MPI_Bsend

MPI_Ssend

```
class mpirun -np 4 -hosts csews1,csews2 ./sendmodes 10000000 1
Rank 3: time= 0.746999
class mpirun -np 4 -hosts csews1,csews2 ./sendmodes 10000000 2
Rank 3: time= 0.746354
class mpirun -np 4 -hosts csews1,csews2 ./sendmodes 10000000 3
Rank 3: time= 0.745962
```

Timing – What to include and when?

```
if (option == 1)
    //standard communication mode
    MPI_Send(arr, BUFSIZE, MPI_INT, size-1, myrank, MPI_COMM_WORLD);
else if (option == 2) {
    buf = (char *)malloc( bufsize );
    MPI_Buffer_attach(buf, bufsize);
    //buffered communication mode
    MPI_Bsend(arr, BUFSIZE, MPI_INT, size-1, myrank, MPI_COMM_WORLD);
}
else if (option == 3)
    //synchronous communication mode
    MPI_Ssend(arr, BUFSIZE, MPI_INT, size-1, myrank, MPI_COMM_WORLD);
```


Performance of Send Modes

```
class mpirun -np 4 -hosts csews1,csews2 ./sendmodes 1000 1
Rank 3: time= 0.000509
class mpirun -np 4 -hosts csews1,csews2 ./sendmodes 1000 2
Rank 3: time= 0.000530
class mpirun -np 4 -hosts csews1,csews2 ./sendmodes 1000 3
Rank 3: time= 0.006357
```

MPI_Send

MPI_Bsend

MPI_Ssend

```
class mpirun -np 4 -hosts csews1,csews2 ./sendmodes 10000000 1
Rank 3: time= 0.746999
class mpirun -np 4 -hosts csews1,csews2 ./sendmodes 10000000 2
Rank 3: time= 0.746354
class mpirun -np 4 -hosts csews1,csews2 ./sendmodes 10000000 3
Rank 3: time= 0.745962
```

Rendezvous

Explicit buffering

Explicit synchronization

```
class mpirun -np 4 -hosts csews1,csews2,csews3,csews4 ./sendmodes 10000000 1
Rank 3: time= 1.113986
class mpirun -np 4 -hosts csews1,csews2,csews3,csews4 ./sendmodes 10000000 2
Rank 3: time= 1.080487
class mpirun -np 4 -hosts csews1,csews2,csews3,csews4 ./sendmodes 10000000 3
Rank 3: time= 1.091517
```

Safety

0

1

MPI_Ssend
MPI_Ssend

MPI_Recv
MPI_Recv

Safe

MPI_Ssend
MPI_Recv

MPI_Ssend
MPI_Recv

Unsafe/Deadlock

MPI_Ssend
MPI_Recv

MPI_Recv
MPI_Ssend

Safe

Note:

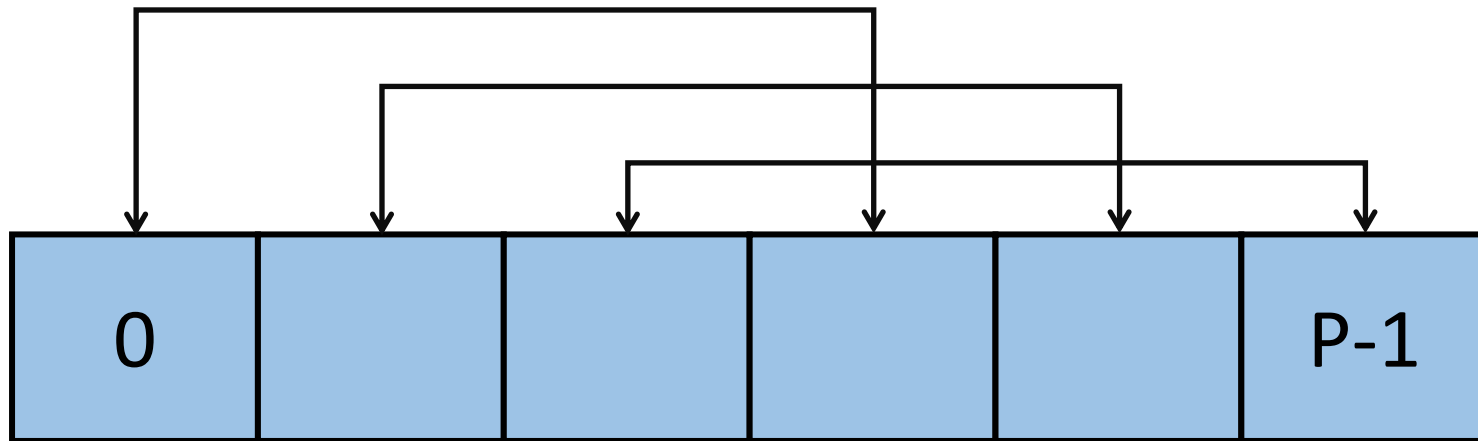
0 sends to 1 and 1 sends to 0

0 receives from 1 and 1 receives from 0

Send-Recv Example ($2 * P/2$ pairwise communications)

Figure shows the **communication pattern**

0 sends to 3 and receives from 3
1 sends to 4 and receives from 4
2 sends to 5 and receives from 5

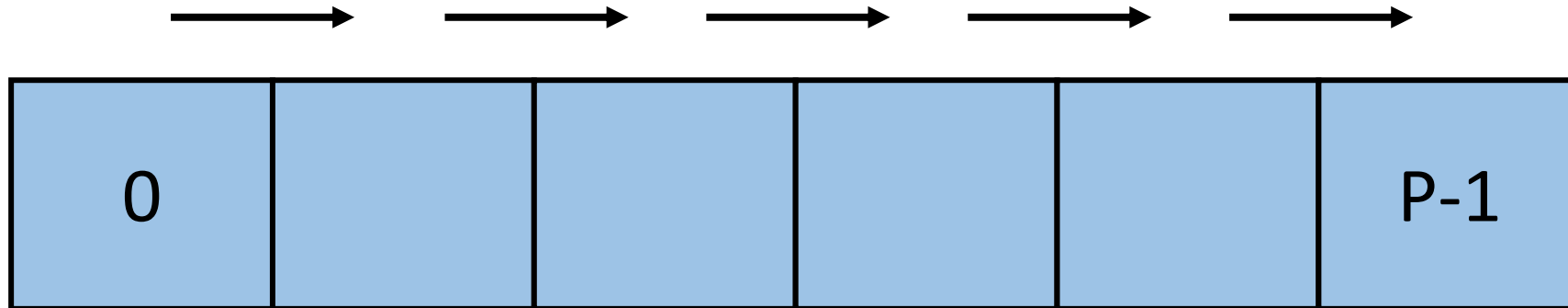


- How many destinations does each process have?
- How many senders does each process have?
- A process communicates with which process?

Code (one of the versions)

```
if (myrank < P/2) {  
    MPI_Send (data, count, MPI_INT, myrank+P/2, myrank+P/2,  
MPI_COMM_WORLD);  
    MPI_Recv (recvbuf, count, MPI_INT, myrank+P/2, myrank, MPI_COMM_WORLD,  
&status);  
}  
else {  
    MPI_Recv (recvbuf, count, MPI_INT, myrank-P/2, myrank, MPI_COMM_WORLD,  
&status);  
    MPI_Send (data, count, MPI_INT, myrank-P/2, myrank-P/2, MPI_COMM_WORLD);  
}
```

Nearest Neighbor (NN) 1D Exchange



Arrows show the **communication pattern**